Unit:3

Knowledge Representation

What is Knowledge Representation?

- **Knowledge Representation** in AI describes the representation of knowledge.
- Basically, it is a study of how the **beliefs, intentions**, and **judgments** of an **intelligent agent** can be expressed suitably for automated reasoning.
- One of the primary purposes of Knowledge Representation includes modeling intelligent behavior for an agent.
- Knowledge Representation and Reasoning (**KR, KRR**) represents information from the real world for a computer to understand and then utilize this knowledge to solve **complex real-life problems** like communicating with human beings in natural language.
- Knowledge representation in AI is not just about storing data in a database, it allows a machine to learn from that knowledge and behave intelligently like a human being.

Knowledge Representation

Following are the kind of knowledge which needs to be represented in AI systems:

Object: All the facts about objects in our world domain. E.g., Guitars contains strings, trumpets are brass instruments.

Events: Events are the actions which occur in our world.

Performance: It describe behavior which involves knowledge about how to do things.

Meta-knowledge: It is knowledge about what we know.

Facts: Facts are the truths about the real world and what we represent.

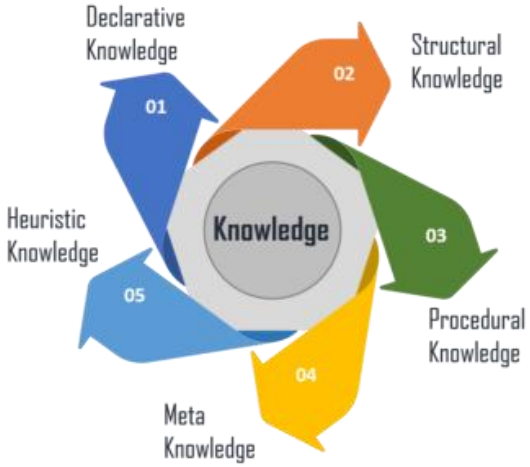
Knowledge-Base: The central component of the knowledge-based agents is the knowledge base. It is represented as KB. The Knowledgebase is a group of the Sentences (Here, sentences are used as a technical term and not identical with the English language).

Knowledge: Knowledge is awareness or familiarity gained by experiences of facts, data, and situations. Following are the types of knowledge in artificial intelligence:

Knowledge Representation

There are 5 types of Knowledge such as:

- **Declarative Knowledge** – It includes concepts, facts, and objects and expressed in a declarative sentence.
- **Structural Knowledge** – It is a basic problem-solving knowledge that describes the relationship between concepts and objects.
- **Procedural Knowledge** – This is responsible for knowing how to do something and includes rules, strategies, procedures, etc.
- **Meta Knowledge** – Meta Knowledge defines knowledge about other types of Knowledge.
- **Heuristic Knowledge** – This represents some expert knowledge in the field or subject.



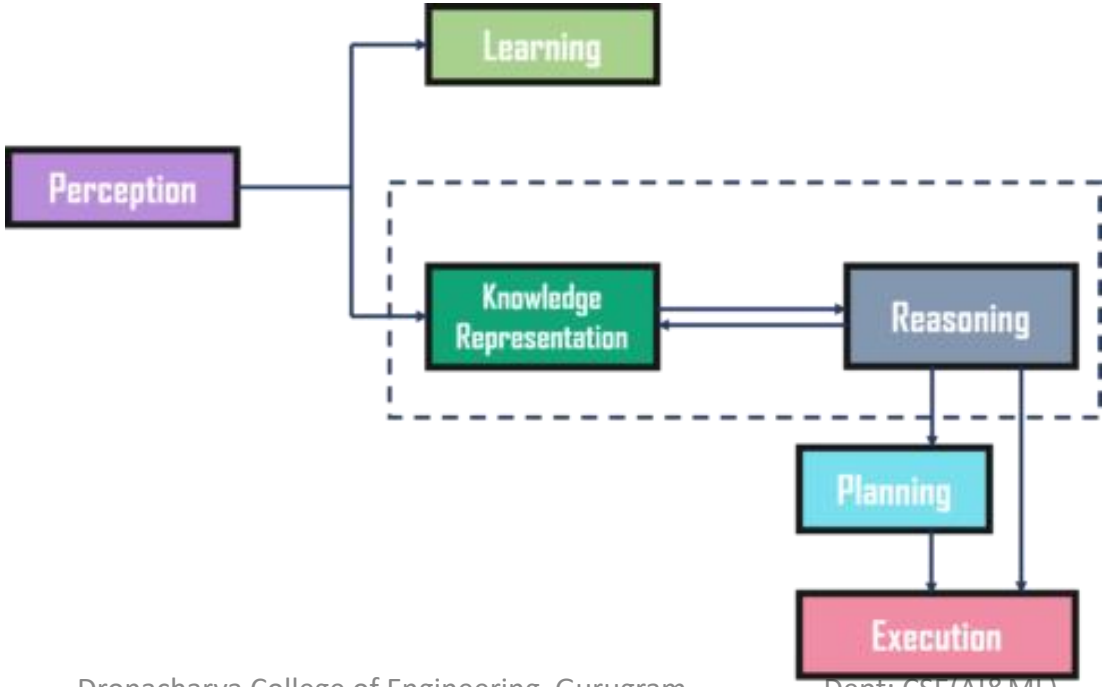
Knowledge Representation

Cycle of Knowledge Representation in AI

Artificial Intelligent Systems usually consist of various components to display their intelligent behavior. Some of these components include:

- Perception
- Learning
- Knowledge Representation & Reasoning
- Planning
- Execution

Example

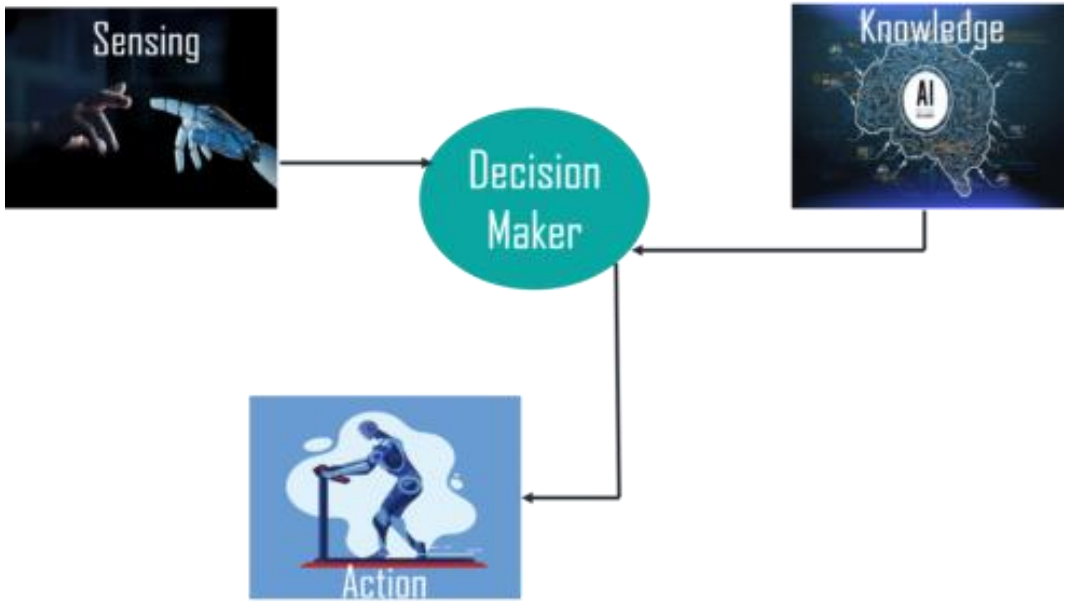


Knowledge Representation

Relation between Knowledge & Intelligence

- In the real world, knowledge plays a vital role in intelligence as well as creating artificial intelligence.
- It demonstrates the intelligent behavior in **AI agents or systems**.
- It is possible for an agent or system to act accurately on some input only when it has the knowledge or experience about the input.

Example



Knowledge Representation

Techniques of Knowledge Representation in AI

- There are four techniques of representing knowledge such as:

Logical Representation

- Logical representation is a language with some **definite rules** which deal with propositions and has no ambiguity in representation.
- It represents a conclusion based on various conditions and lays down some important **communication rules**.
- Also, it consists of precisely defined syntax and semantics which supports the sound inference.
- Each sentence can be translated into logics using syntax and semantics.

Syntax	Semantics
•It decides how we can construct legal sentences in logic. •It determines which symbol we can use in knowledge representation. •Also, how to write those symbols.	•Semantics are the rules by which we can interpret the sentence in the logic. •It assigns a meaning to each sentence.

Logical Representation

Advantages:

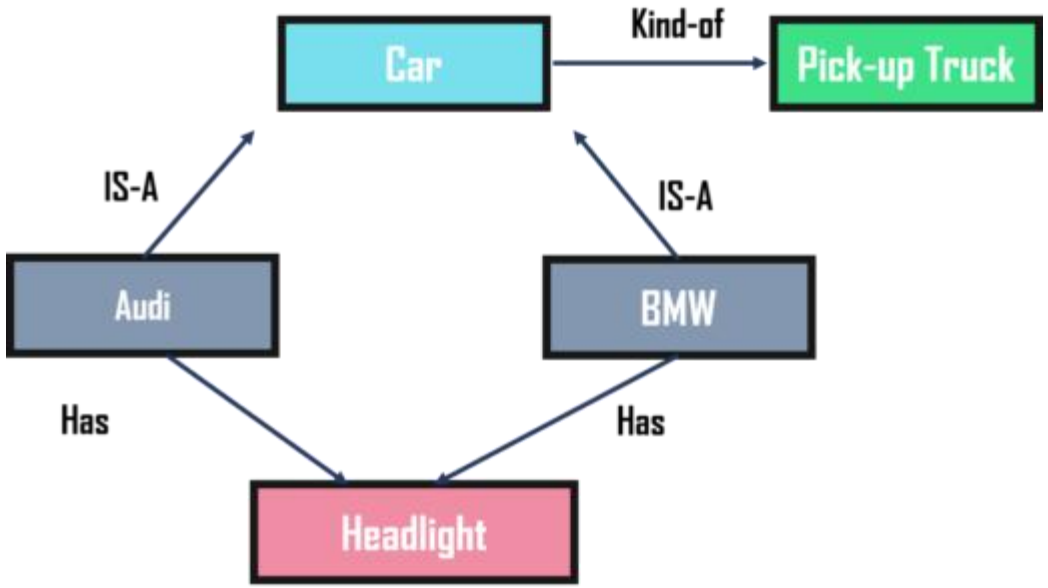
- Logical representation helps to perform logical reasoning.
- This representation is the basis for the programming languages.

Disadvantages:

- Logical representations have some restrictions and are challenging to work with.
- This technique may not be very natural, and inference may not be very efficient.

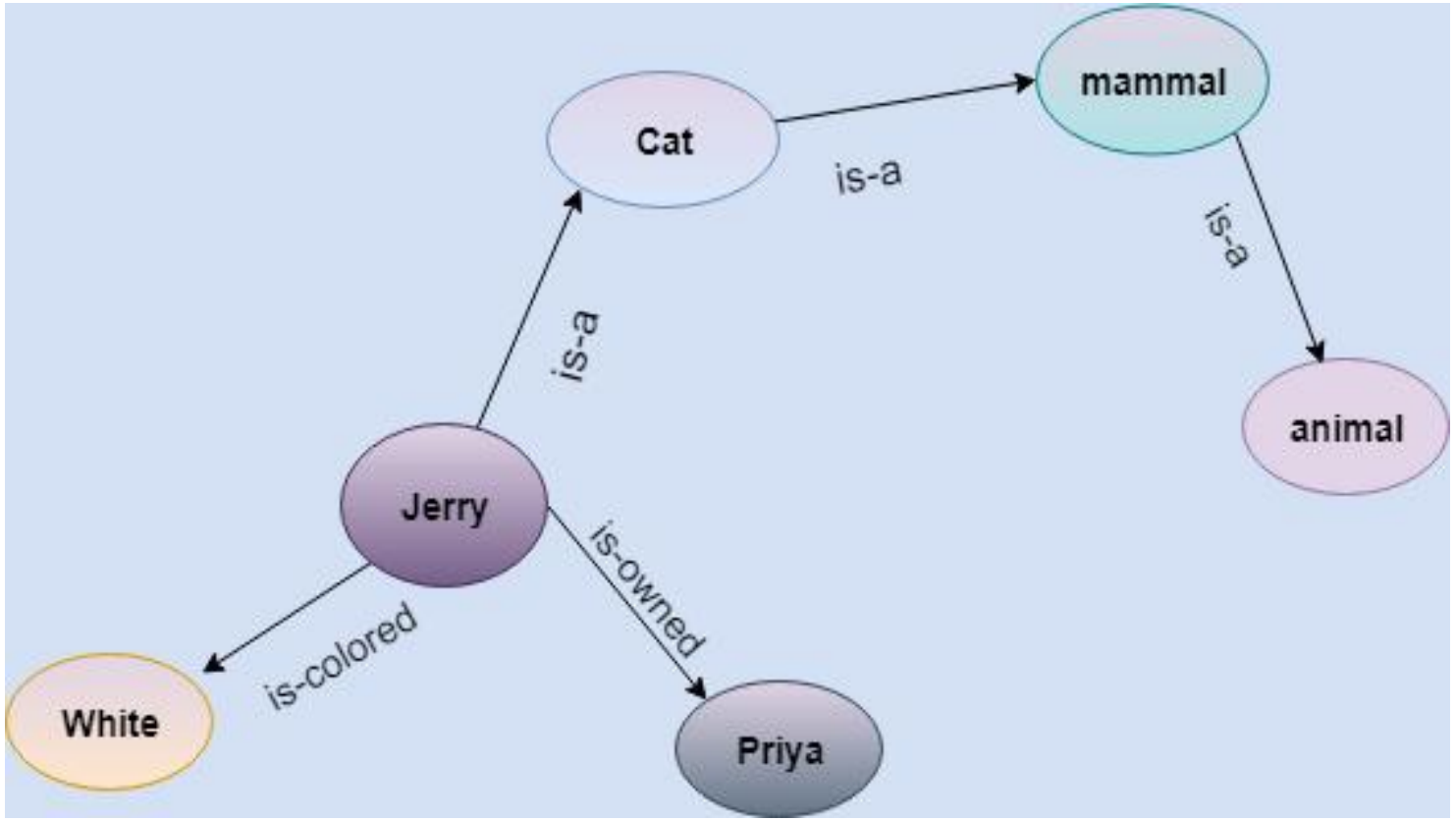
Semantic Network Representation

- Semantic networks work as an **alternative** of **predicate logic** for knowledge representation.
- **These describe both objects and events in general**
- In Semantic networks, we can represent our knowledge in the form of graphical networks. This network consists of nodes representing objects and arcs which describe the relationship between those objects. Also, it categorizes the object in different forms and links those objects.



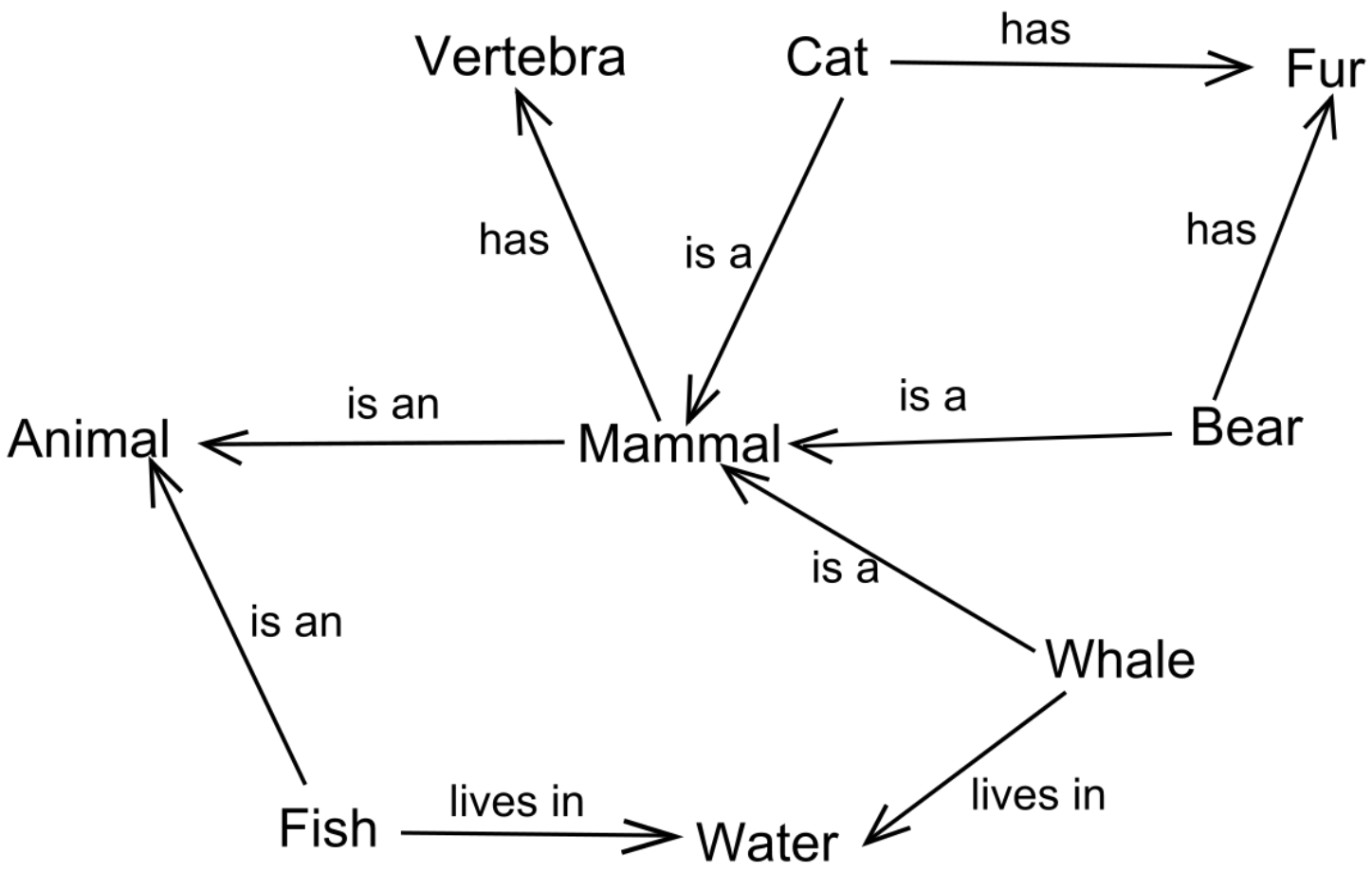
Semantic Network Representation

Example



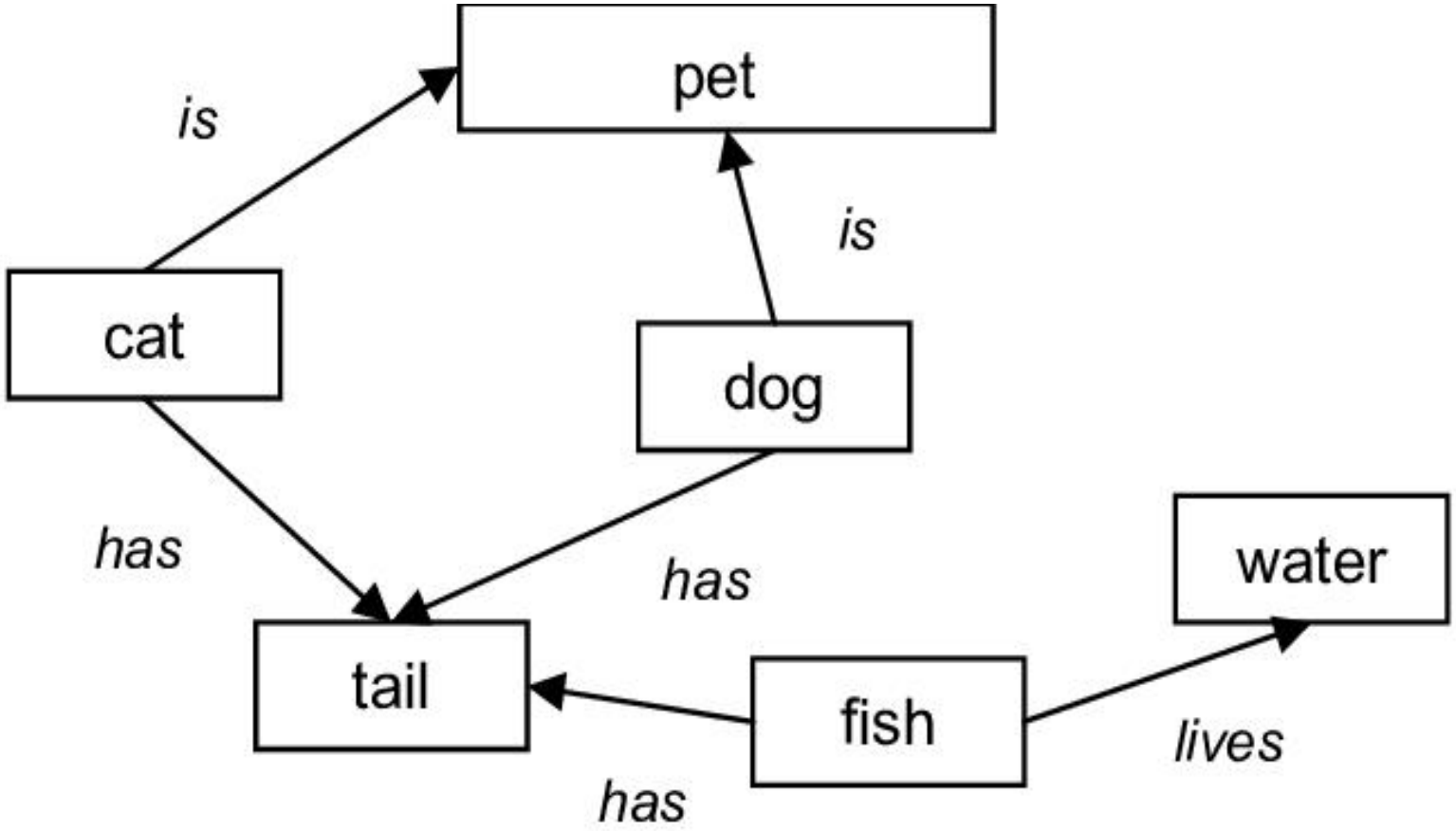
Semantic Network Representation

Example



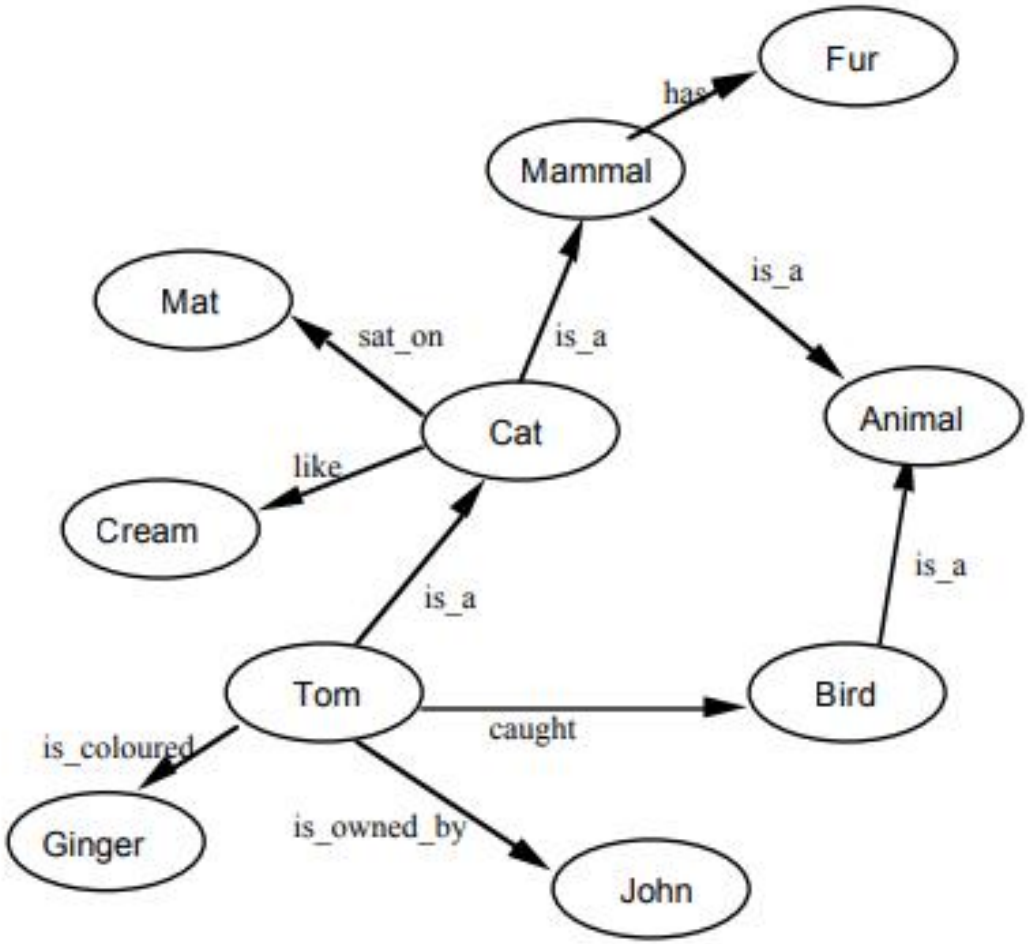
Semantic Network Representation

Example



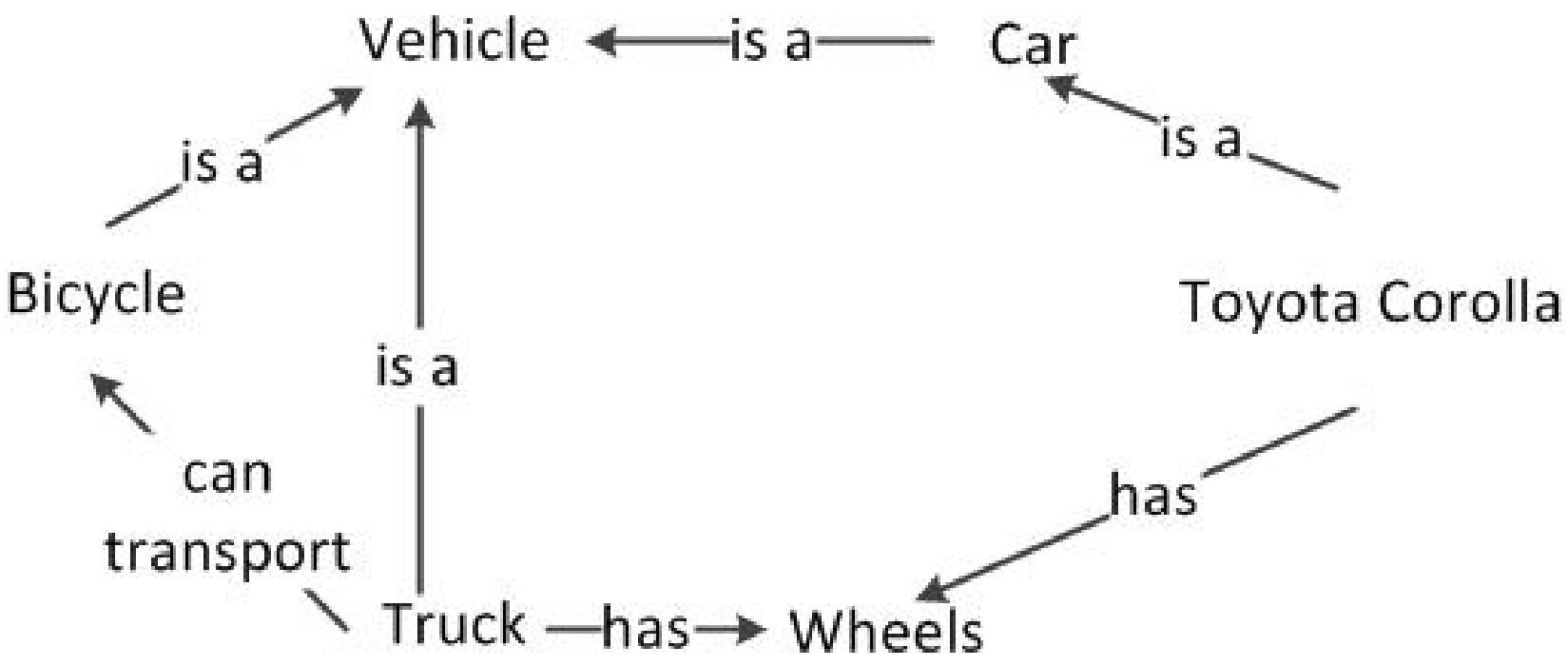
Semantic Network Representation

Example



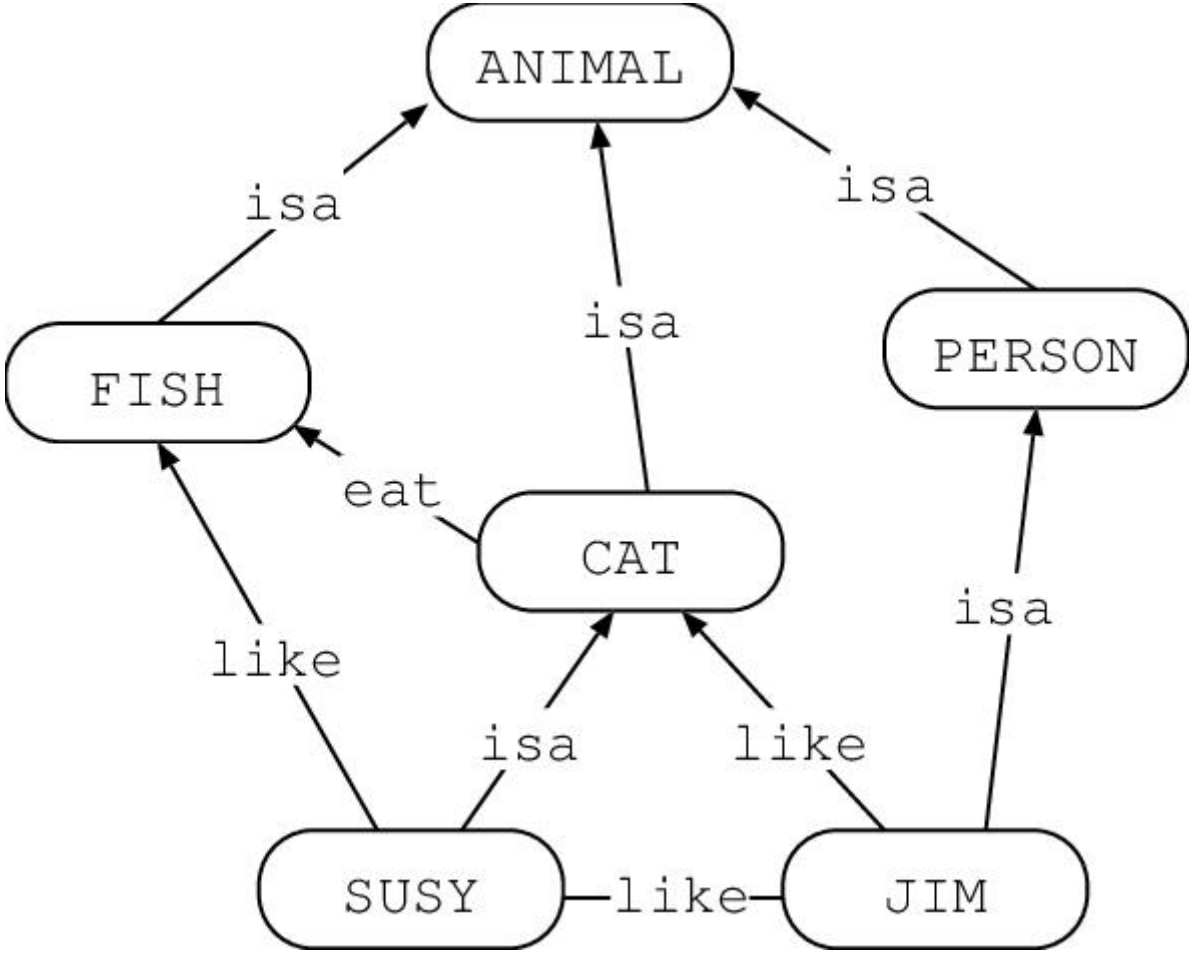
Semantic Network Representation

Example



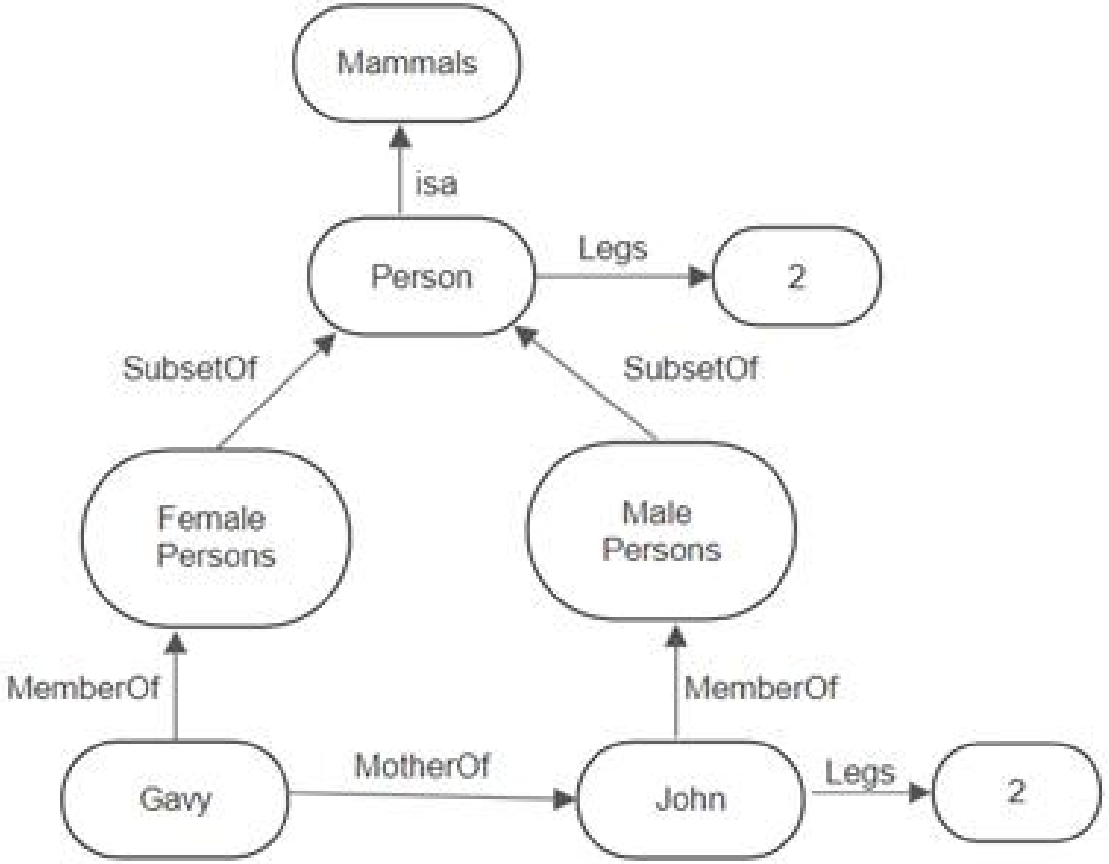
Semantic Network Representation

Example



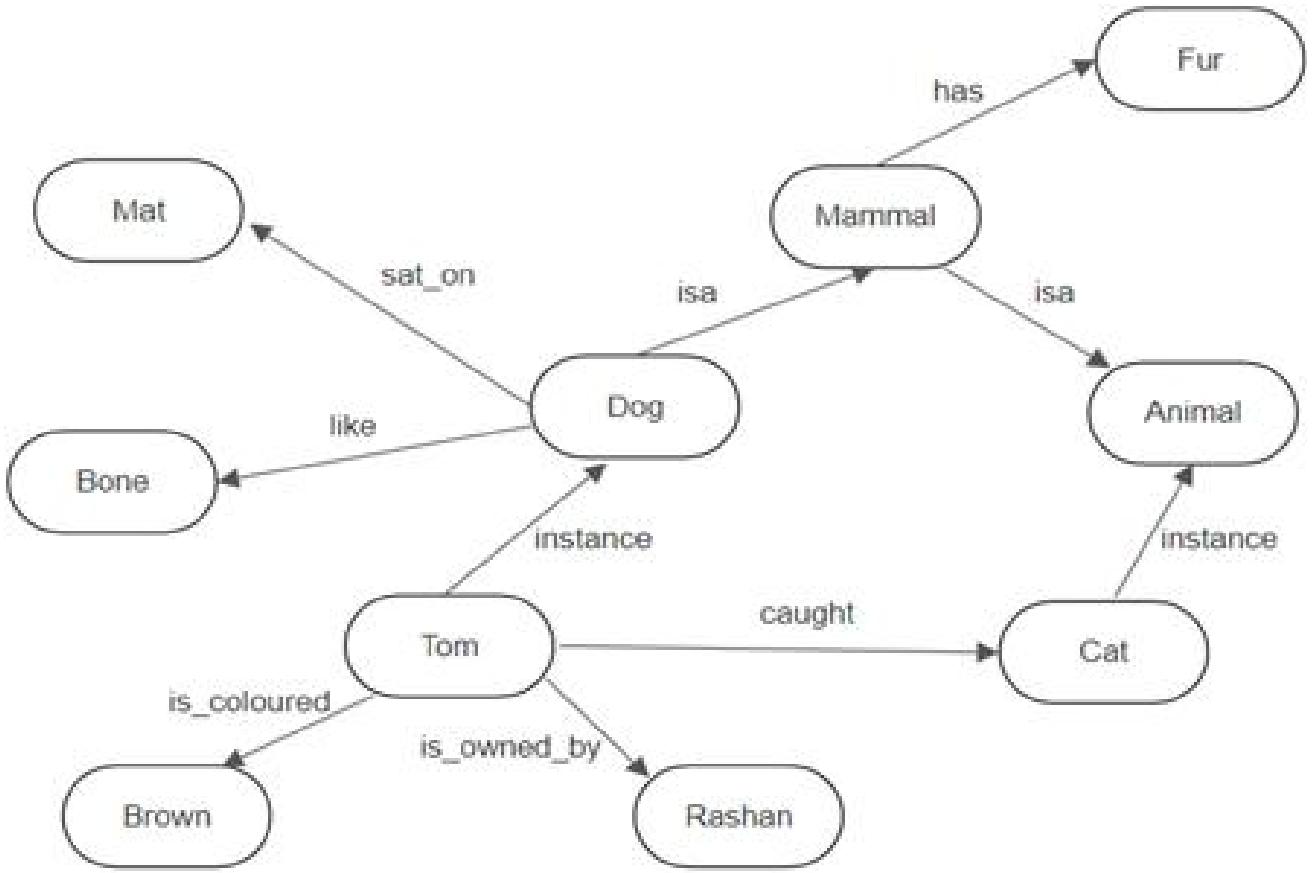
Semantic Network Representation

Example



Semantic Network Representation

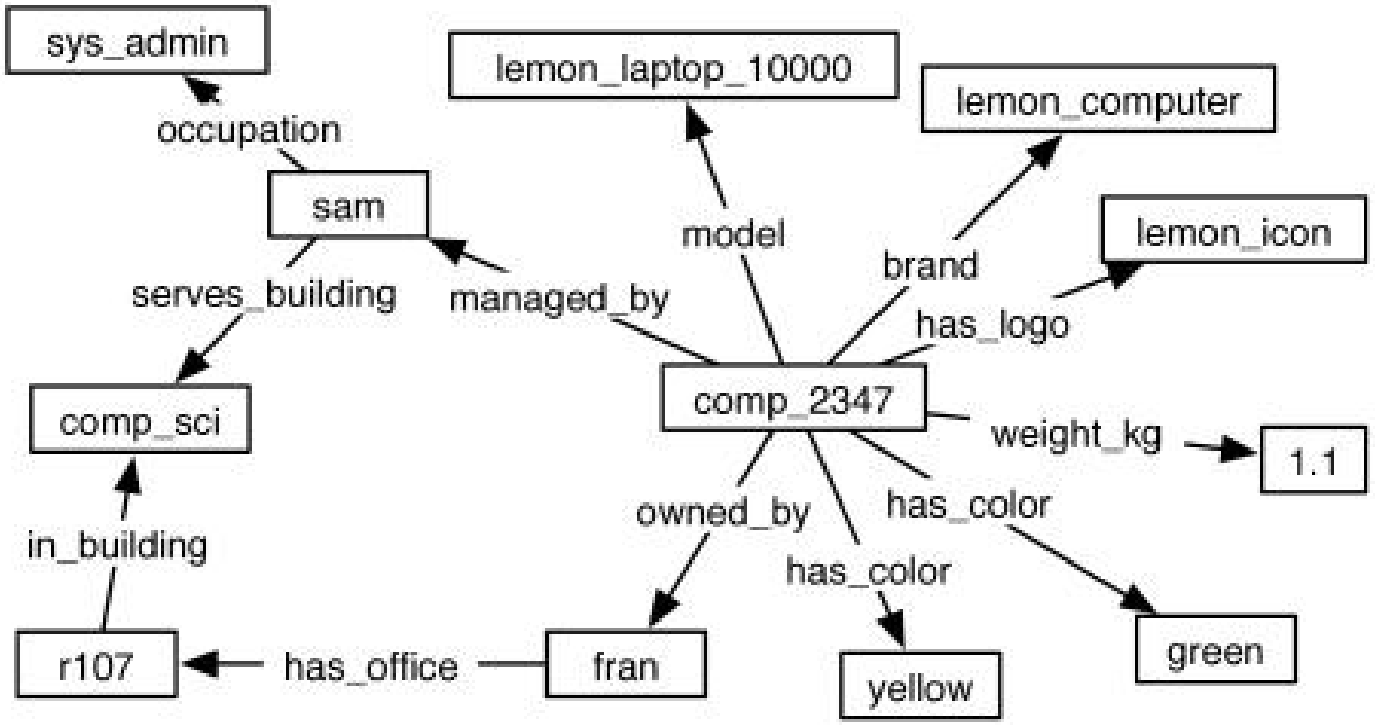
Example



Semantic Network Representation

Example

Figure shows a semantic network for the delivery robot showing the sort of knowledge that the robot might have about a particular computer in a university department.



Semantic Network Representation

Advantages:

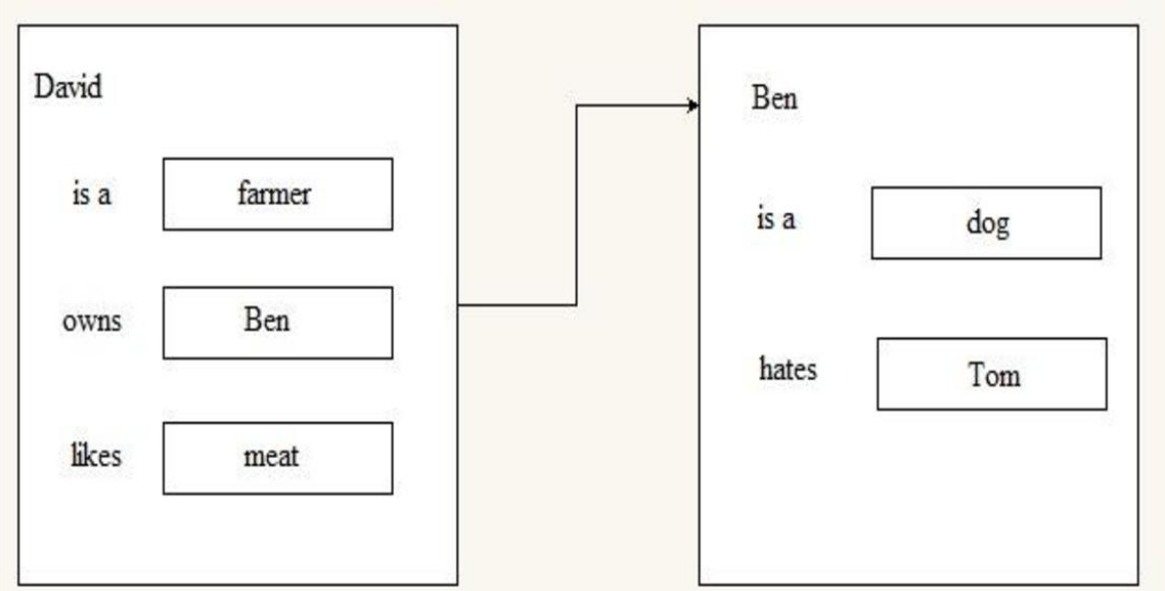
- Semantic networks are a natural representation of knowledge.
- Also, it conveys meaning in a transparent manner.
- These networks are simple and easy to understand.

Disadvantages:

- Semantic networks take more computational time at runtime.
- Also, these are inadequate as they do not have any equivalent quantifiers.
- These networks are not intelligent and depend on the creator of the system.

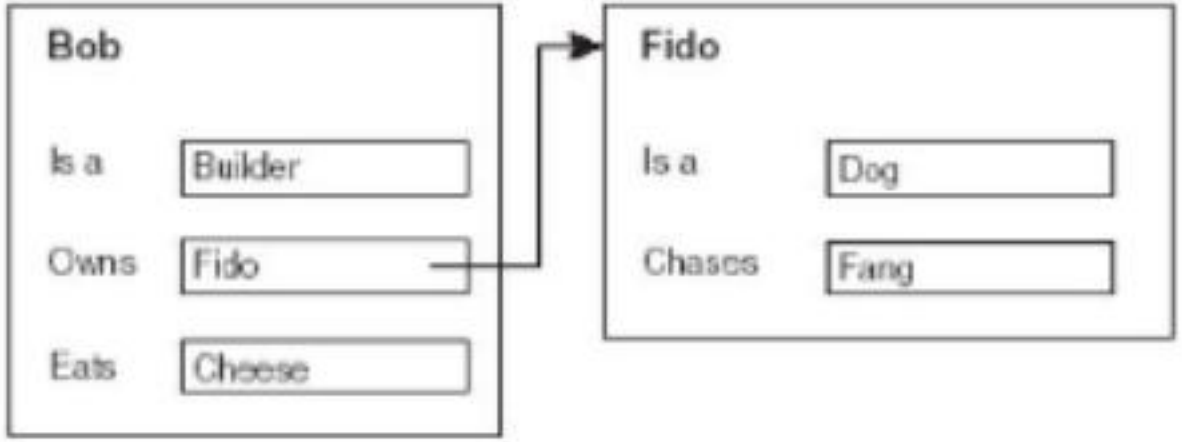
Frame Representation

- A frame is a **record** like structure that consists of a **collection of attributes** and values to describe an entity in the world.
- These are the AI data structure that divides knowledge into substructures by representing stereotypes situations.
- Basically, it consists of a collection of slots and slot values of any type and size.
- Slots have names and values which are called facets.



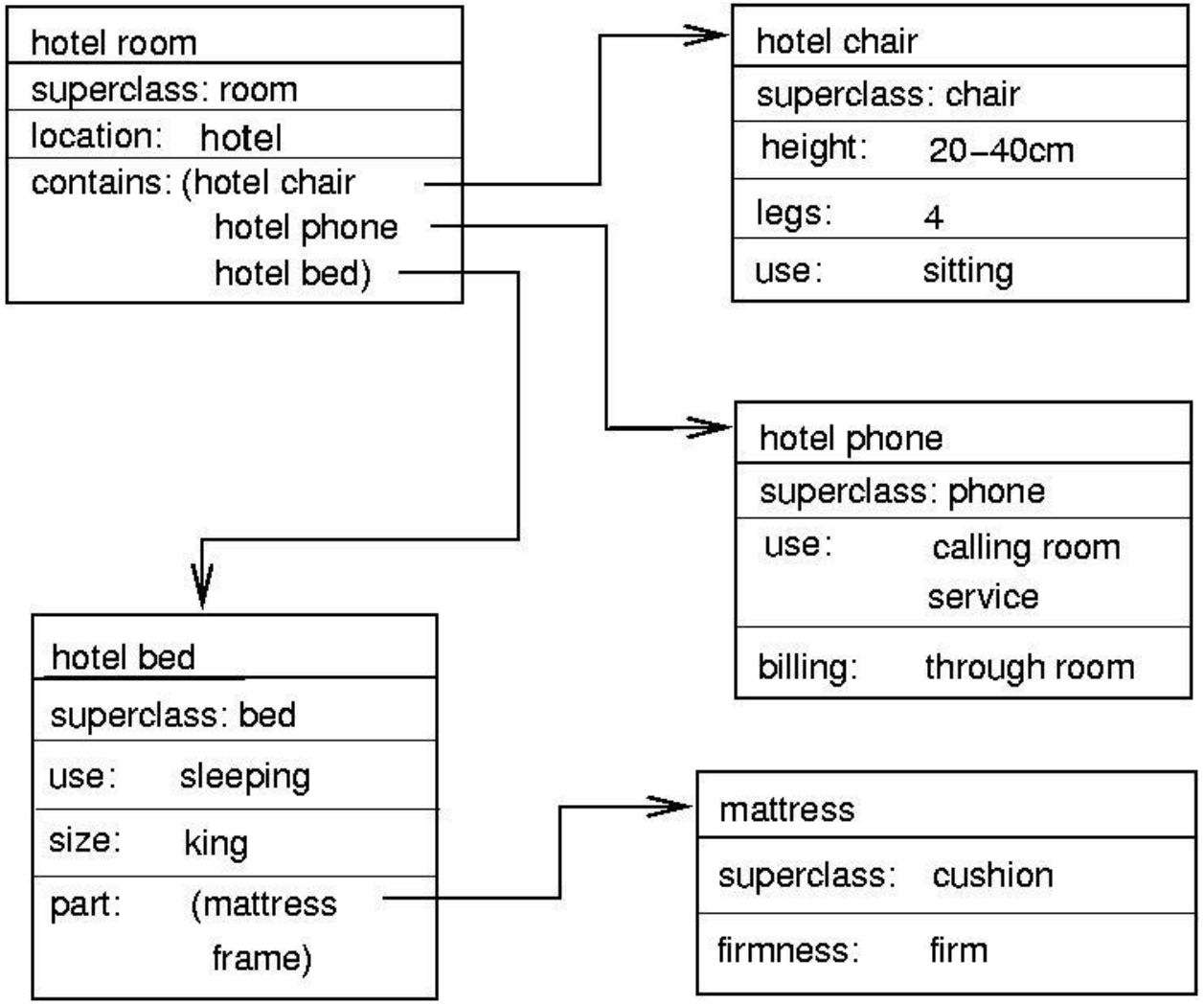
Frame Representation

Example



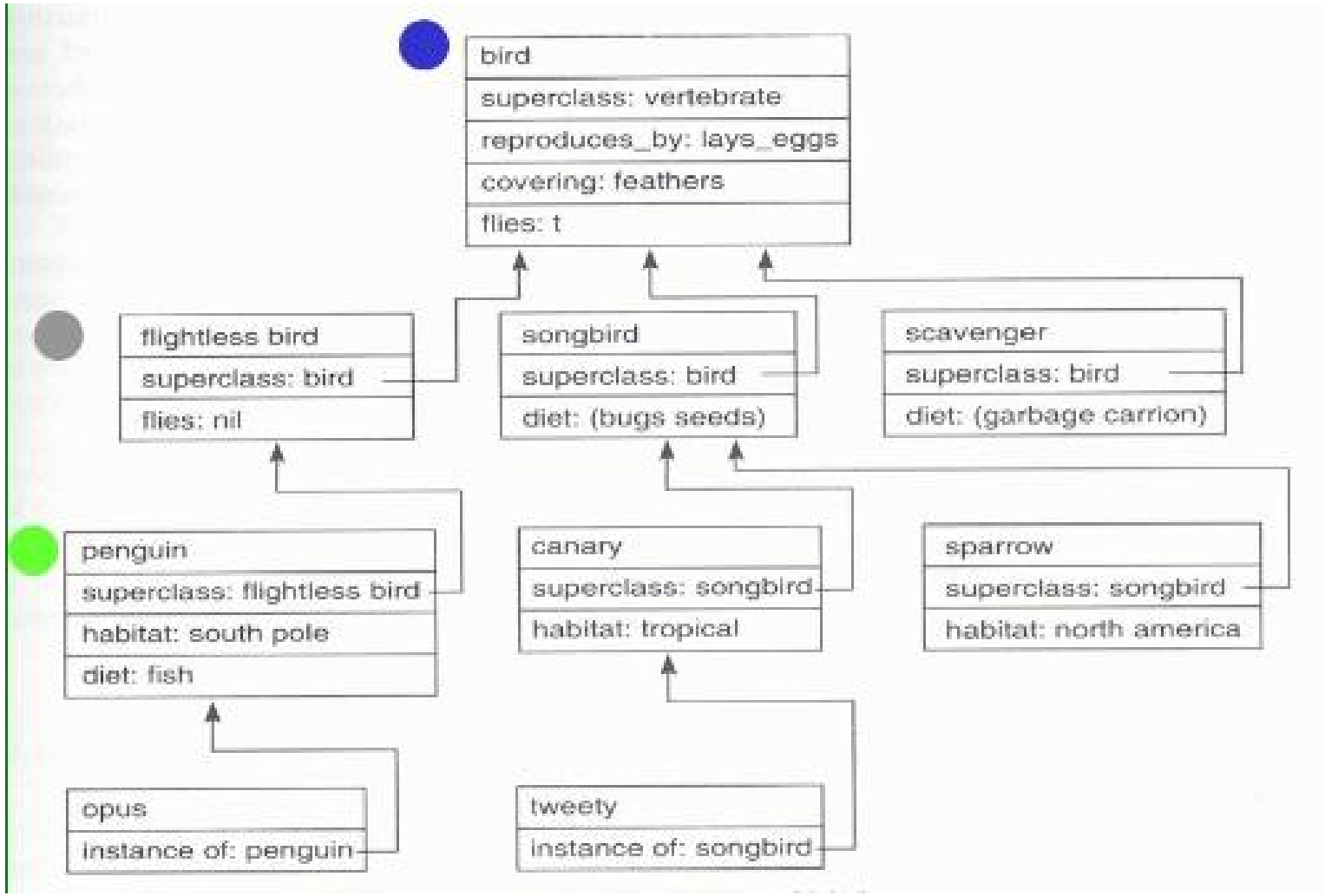
Frame Representation

Example



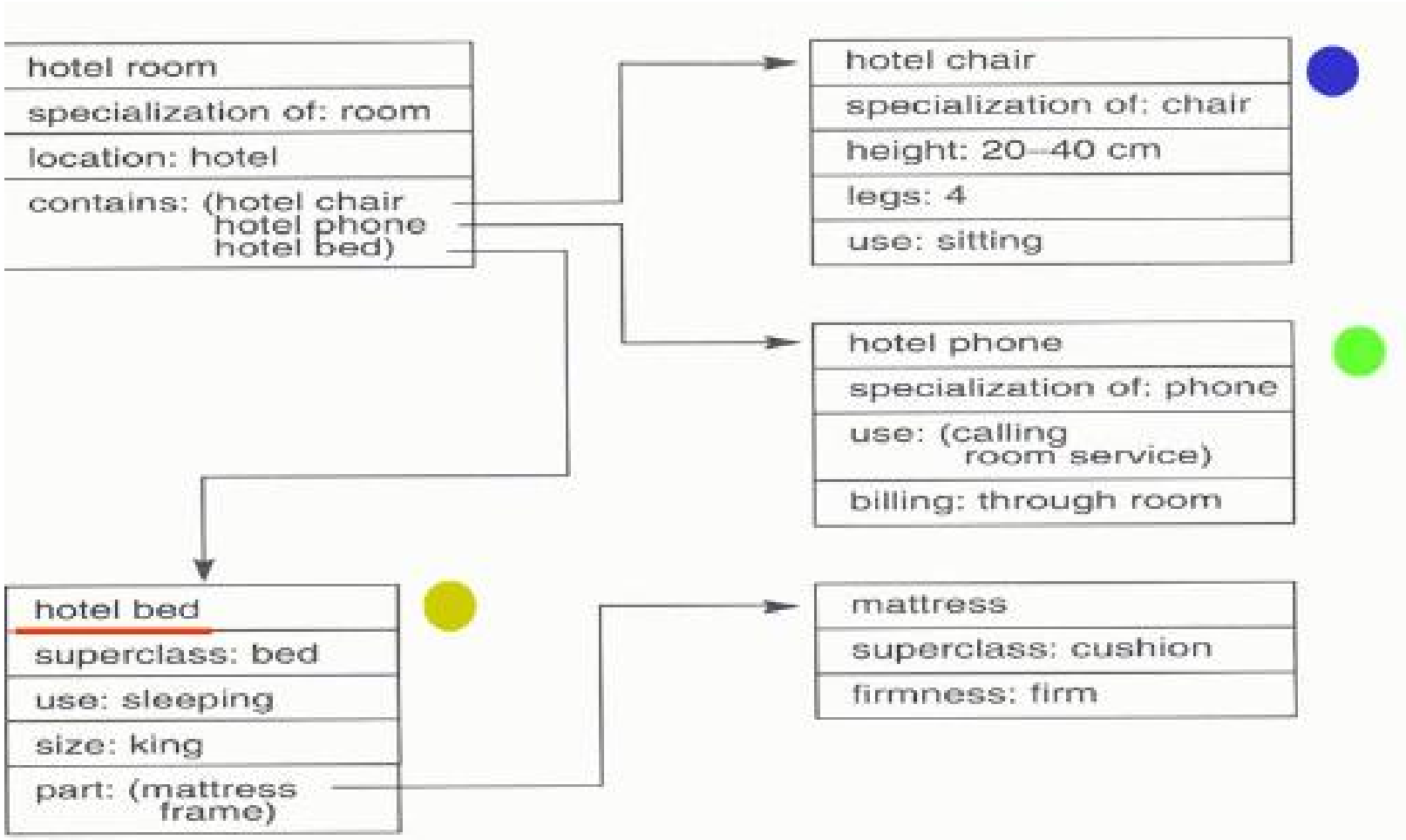
Frame Representation

Example



Frame Representation

Example



Frame Representation

Example: 1

Let's take an example of a frame for a book

Slots	Filters
Title	Artificial Intelligence
Genre	Computer Science
Author	Peter Norvig
Edition	Third Edition
Year	1996

Frame Representation

Advantages:

- It makes the programming easier by grouping the related data.
- Frame representation is easy to understand and visualize.
- It is very easy to add slots for new attributes and relations.
- Also, it is easy to include default data and search for missing values.

Disadvantages:

- In frame system inference, the mechanism cannot be easily processed.
- The inference mechanism cannot be smoothly proceeded by frame representation.
- It has a very generalized approach.

Daily Quiz

Q1 **There exist only two types of quantifiers, Universal Quantification and Existential Quantification.**

a) True

b) False

Answer: a

Q2. **First Order Logic is also known as _____**

a) First Order Predicate Calculus

b) Quantification Theory

c) Lower Order Calculus

d) All of the mentioned

Answer: d

Daily Quiz

Q3. **How many proposition symbols are there in artificial intelligence?**

a) 1

b) 2

c) 3

d) 4

Answer: b

Q4. **Which is also called single inference rule?**

a) Reference

b) Resolution

c) Reform

d) None of the mentioned

Answer: b

MCQs

Q4. **Translate the following statement into FOL.**

"For every a, if a is a philosopher, then a is a scholar"

a) ∀ a philosopher(a) → scholar(a)

b) ∃ a philosopher(a) → scholar(a)

c) All of the mentioned

d) None of the mentioned

Answer: a

Q5. **Which is created by using single propositional symbol?**

a) Complex sentences

b) Atomic sentences

c) Composition sentences

d) None of the mentioned

Answer: b

MCQs

Q6. **Which are needed to compute the logical inference algorithm?**

a) Logical equivalence

b) Validity

c) Satisfiability

d) All of the mentioned

Answer: d

Introduction to Logic

- Logic in Artificial Intelligence is a systematic set of guidelines that specify how information is represented and processed to get favorable outcomes. It offers a framework for thinking critically about the world, making choices, and dealing with issues. Many AI technologies, such as expert systems, knowledge graphs, and decision trees, are built on logic.

- There are mainly two types of logic which are used in Artificial Intelligence.
  - **Deductive logic**
  - **Inductive logic**

Introduction to Logic

Deductive Logic

- It **uses the top-down approach** to find its reasoning, which helps it to decide and make decisions.
- This logic first works with broad premises or guiding principles to find specific findings or forecasts.
- For example, we can use deductive reasoning to determine that Ravi is mortal if we know that all humans are mortal and that Ravi is a human.

Inductive Logic

- inductive logic **uses the bottom-up method** to find its reasoning, which helps it to decide and make decisions. It starts with specific observations or facts, general principles or theories derived to find the outcomes.
- For example, we might use inductive reasoning to conclude that all crows are black because we noticed that every one of them is black.

Introduction to Logic

- Artificial Intelligence makes use of both inductive and deductive logic.

- Deductive logic is used in rule-based and expert systems, where precise rules are employed to generate specific conclusions. But, when we use data to derive broad patterns and principles, inductive logic is used to make machine learning models and data analytics.

Propositional logic (PL)

- Also known as **sentential logic**.
- It is a fundamental component of artificial intelligence (AI).
- Propositional logic (PL) is the simplest form of logic where all the statements are made by propositions.
- A proposition is a declarative statement which is either true or false.
- One of the key applications of propositional logic in AI is in the representation of knowledge.

Example:

- a) It is Sunday.
- b) The Sun rises from West (False proposition)
- c) 3+3= 7(False proposition)
- d) 5 is a prime number.

Propositional logic

Some basic facts about propositional logic:

- Propositional logic is also called Boolean logic as it works on 0 and 1.
- In propositional logic, we use symbolic variables to represent the logic, and we can use any symbol for a representing a proposition, such A, B, C, P, Q, R, etc.
- Propositions can be either true or false, but it cannot be both.
- Propositional logic consists of an object, relations or function, and logical connectives. These connectives are also called logical operators.
- The propositions and connectives are the basic elements of the propositional logic.
- A proposition formula which is always true is called **Tautology**, and it is also called a valid sentence.
- A proposition formula which is always false is called **Contradiction**.
- A proposition formula which has both true and false values is called **Contingency**.
- Statements which are questions, commands, or opinions are not propositions such as "**Where is Rohini**", "**How are you**", "**What is your name**", are not propositions.

Propositional logic

There are two types of Propositions:

Atomic Propositions

Compound propositions

Atomic Proposition:

- Atomic propositions are the simple propositions.
- It consists of a single proposition symbol.
- These are the sentences which must be either true or false.

Example:

a) 2+2 is 4, it is an atomic proposition as it is a **true** fact.

b) "The Sun is cold" is also a proposition as it is a **false** fact.

Propositional logic

Compound proposition:

- Compound propositions are constructed by combining simpler or atomic propositions, using parenthesis and logical connectives.

Example:

a) "It is raining today and street is wet."

b) "Ankit is a doctor and his clinic is in Mumbai."

Logical Connectives:

- Logical connectives are used to connect two simpler propositions or representing a sentence logically.
- We can create compound propositions with the help of logical connectives.
- There are mainly **five** logical connectives

Propositional logic

1. Negation: A statement like ¬P is referred to as a negation of P.

2. Conjunction: A conjunction is a sentence that contains ∧ connective such as, **P ∧ Q**.

Example: Rohan is intelligent and hardworking.

It can be written as,
P= Rohan is intelligent,
Q= Rohan is hardworking

P∧ Q

3. Disjunction: A disjunction is a sentence with a connective ∨ , such as

P ∨ Q, where P and Q are the propositions.

Example: Ritika is a doctor or Engineer
Here,

P= Ritika is Doctor

Q= Ritika is an Engineer

so we can write it as **P ∨ Q**.

Propositional logic

4. Implication: An implication is a statement such as P → Q. **If-then rules** are another name for implications. It can be expressed as follows:

If it is raining, **then** the street is wet.

Let P= It is raining, and Q= Street is wet, so it is represented as **P →Q**

5. Bi-conditional: A sentence such as P⇔ Q is a Bi-conditional sentence,

Example: If I am breathing, then I am alive

Let, P= I am breathing, Q= I am alive, it can be represented as P ⇔ Q.

Propositional logic

Propositional Logic Connectives:

Connective symbols	Word	Technical term	Example
$\wedge$	AND	Conjunction	$A \wedge B$
$\vee$	OR	Disjunction	$A \vee B$
$\rightarrow$	Implies	Implication	$A \rightarrow B$
$\Leftrightarrow$	If and only if	Biconditional	$A \Leftrightarrow B$
$\neg$ or $\sim$	Not	Negation	$\neg A$ or $\neg B$

Propositional logic

Truth Table

We can combine all the possible combination with logical connectives, and the representation of these combinations in a tabular format is called **Truth table**.

For Negation:

P	¬ P
True	**False**
False	**True**

For Conjunction:

P	Q	P∧ Q
True	True	**True**
True	False	**False**
False	True	**False**
False	False	**False**

Propositional logic

Truth Table

For disjunction:

P	Q	P V Q.
True	True	**True**
False	True	**True**
True	False	**True**
False	False	**False**

For Implication:

P	Q	P→ Q
True	True	**True**
True	False	**False**
False	True	**True**
False	False	**True**

For Biconditional:

P	Q	P⇔ Q
True	True	**True**
True	False	**False**
False	True	**False**
False	False	**True**

Propositional logic

Precedence of connectives:

Precedence	Operators
First Precedence	Parenthesis
Second Precedence	Negation
Third Precedence	Conjunction(AND)
Forth Precedence	Disjunction(OR)
Fifth Precedence	Implication
Sixth Precedence	Biconditional

Propositional logic

Logical equivalence:

- Two propositions are said to be logically equivalent if and only if the columns in the truth table are identical to each other.
- In below truth table we can see that column for ¬A ∨ B and A→B, are identical.

A	B	¬A	¬A∨ B	A→B
T	T	F	T	T
T	F	F	F	F
F	T	T	T	T
F	F	T	T	T

Propositional logic

Properties of Operators:

1. Commutativity:

$P \wedge Q = Q \wedge P$, or

$P \vee Q = Q \vee P$.

2. Associativity:

$(P \wedge Q) \wedge R = P \wedge (Q \wedge R)$,

$(P \vee Q) \vee R = P \vee (Q \vee R)$

3. Identity element:

$P \wedge True = P$,

$P \vee True = True$.

4. Distributive:

$P \wedge (Q \vee R) = (P \wedge Q) \vee (P \wedge R)$.

$P \vee (Q \wedge R) = (P \vee Q) \wedge (P \vee R)$.

5. De-Morgan's Law:

$\neg (P \wedge Q) = (\neg P) \vee (\neg Q)$

$\neg (P \vee Q) = (\neg P) \wedge (\neg Q)$.

6. Double-negation elimination:

$\neg (\neg P) = P$.

Propositional logic

Example. Construct a truth table for $(P \to Q) \wedge (Q \to R)$

P	Q	R	$P \to Q$	$Q \to R$	$(P \to Q) \wedge (Q \to R)$
T	T	T	T	T	T
T	T	F	T	F	F
T	F	T	F	T	F
T	F	F	F	T	F
F	T	T	T	T	T
F	T	F	T	F	F
F	F	T	T	T	T
F	F	F	T	T	T

□

Propositional logic

Example. Show that $P \rightarrow Q$ and $\neg P \vee Q$ are logically equivalent.

P	Q	$P \rightarrow Q$	$\neg P$	$\neg P \vee Q$
T	T	T	F	T
T	F	F	F	F
F	T	T	T	T
F	F	T	T	T

Propositional logic

Example. Construct truth tables to check the following sentences for tautology, contradiction and contingency.

1. $(P \vee \sim P) \wedge (Q \wedge \sim Q)$
2. $(P \wedge \sim P) \vee (Q \wedge \sim Q)$
3. $P \vee (Q \rightarrow P)$

Propositional logic

Limitations of Propositional logic:

- We cannot represent relations like **ALL**, **some**, or **none** with propositional logic.

  Example:

  All the girls are intelligent.

  Some apples are sweet.

- Propositional logic has limited expressive power.
- In propositional logic, we cannot describe statements in terms of their properties or logical relationships

Propositional logic

P	Q	P → Q	Q → P	¬ Q → ¬ P	¬ P → ¬ Q.
T	T	T	T	T	T
T	F	F	T	F	T
F	T	T	F	T	F
F	F	T	T	T	T

Hence from the above truth table, we can prove that P → Q is equivalent to ¬ Q → ¬ P, and Q→ P is equivalent to ¬ P → ¬ Q.

Rules of Inference in AI

- Inference in AI refers to the process of reasoning and making decisions based on available information or data.
- It involves deriving new knowledge or conclusions from existing knowledge or data.
- Inference is used in many AI applications, including natural language processing, computer vision, robotics, and expert systems.
- By using rules of inference, machines can analyze complex data and draw conclusions based on logical relationships between pieces of information.
- These rules provide a foundation for automated reasoning and form a critical part of the AI toolkit for solving real-world problems.
- In AI, inference can be categorized into two types: deductive inference and inductive inference.
- Deductive inference involves reasoning from general principles to specific conclusions, while inductive inference involves inferring general principles or rules based on specific observations or data.

Types of Inference Rules in AI

- Modus Ponens
- Modus Tollens
- Hypothetical Syllogism
- Disjunctive Syllogism
- Addition
- Simplification
- Resolution

Types of Inference Rules in AI

Modus Ponens

- It is a deductive inference rule in which if A implies B and A is true, then B must also be true.
- It is also known as affirming the antecedent.
- Example, "If it's raining, then the ground is wet" (A implies B), "It's raining" (A is true), therefore "The ground is wet" (B is true).

Symbolic Notation: $(P \rightarrow Q), P \vdash Q$

Explanation:
The symbol "→" means "implies" or "if...then".
So ($P \rightarrow Q$) means "if P, then Q". The symbol "$\vdash$ " means "entails" or "leads to", and is used to indicate that Q can be deduced from the premises (P → Q) and P.

Types of Inference Rules in AI

Modus Tollens

- It is a deductive inference rule in which if A implies B and B is false, then A must also be false.
- It is also known as denying the consequent.
- Example, "If it's raining, then the ground is wet" (A implies B), "The ground is not wet" (B is fasle), therefore "It's not raining" (A is false).

Symbolic Notation: $(P \rightarrow Q), \neg Q \vdash \neg P$

Explanation:

The symbol "¬" means "not". So **¬Q** means "It is not the case that Q".

The symbol "⊢ " means "entails" or "leads to".

This rule can be read as "If P implies Q and Q is false, then P is false".

Types of Inference Rules in AI

Hypothetical Syllogism

- It is a deductive inference rule that allows us to conclude from two conditional statements.
- If A implies B and B implies C, then A implies C.
- Example, "If it's raining, then the ground is wet" (A implies B), "If the ground is wet, then the grass will be green" (B implies C), therefore "If it's raining, then the grass will be green" (A implies C).

Symbolic Notation: $(P \rightarrow Q), (Q \rightarrow R) \vdash (P \rightarrow R)$

Explanation:
This rule says that if P implies Q, and Q implies R, then P implies R. The symbol "→" means "implies", and the symbol "⊢ " means "entails".

Types of Inference Rules in AI

Disjunctive Syllogism

- It is a deductive inference rule that allows us to conclude a disjunctive statement.
- If it's either A or B and A is false, then B must be true.
- Example, "It's either raining or snowing" (A or B), "It's not raining" (A is false), therefore "It's snowing" (B is true).

Symbolic Notation: $(P \vee Q), \neg P \vdash Q$

Explanation:
The symbol "$\vee$" means " or", so ($P \vee Q$) means "either P or Q (or both) is true".
The symbol "¬" means "not". This rule can be read as "If either P or Q is true, and P is false, then Q is true".

Types of Inference Rules in AI

Addition

- It is a deductive inference rule that allows us to add a statement to conjunction (A conjunction implies that both statements are true).
- If A is true, then A or B is also true. For example, "It's raining" (A is true), therefore "It's either raining or snowing" (A or B is true).

Symbolic Notation: P ├ (P V Q)

Explanation:
This rule says that if P is true, then P or Q is true. The symbol " V " means " or".

Types of Inference Rules in AI

Simplification

- It is a deductive inference rule that allows us to derive a simpler statement from conjunction.
- If A and B are true, then A is true.
- Example, "It's raining and the ground is wet" (A and B are true), therefore "It's raining" (A is true).

Symbolic Notation: $(P \land Q) \vdash P$

Explanation:
The symbol "$\land$" means " and", so ($P \land Q$) means "both P and Q are true". This rule can be read as "If both P and Q are true, then P is true".

Types of Inference Rules in AI

Resolution

- It is a deductive inference rule that allows us to resolve a disjunction(disjunction implies that at least one statement is true).
- If A implies B and C implies not B, then A implies not C.
- Example, "If it's raining, then the ground is wet" (A implies B), "If the ground is not wet, then it's not raining" (C implies not B), therefore "If it's raining, then the ground is not dry" (A implies not C).

Symbolic Notation: $(P \vee Q), (\neg P \vee R) \vdash (Q \vee R)$

Explanation:
This rule can be read as "If either P or Q is true, and not-P or R is true, then Q or R is true". The symbol "¬" means "not", and the symbol "∨" means "or".

First-Order Logic in Artificial intelligence

- The propositional logic has very limited expressive power.
- In propositional logic, we can only represent the facts, which are either true or false.
- Propositional logic is not sufficient to represent the complex sentences or natural language statements.
- The following sentences are not possible to represent using propositional logic:
  - **Some humans are intelligent**
  - **I love mankind. It's the people I can't stand!**
  - **Joe Root likes football.**
  - **I like to eat mangos.**
- **First Order Logic** in Artificial Intelligence (also known as **Predicate Logic** or **First Order Predicate Logic**) is an extension of propositional logic and unlike propositional logic, it is sufficiently expressive in representing any natural language construct.
- First Order Logic is a technique used for **knowledge representation**.
- It is a robust technique to represent objects as well as their relationships.

First-Order Logic in Artificial intelligence

Unlike propositional logic, First Order Logic in Artificial Intelligence doesn't only include facts but also different other entities as listed below:

1. **Objects:** Objects can denote any real-world entity or any variable. E.g., A, B, colors, theories, circles etc.
2. **Relations:** Relations represent the links between different objects. Relations can be
   - **unary** (relations defined for a single term)

     E.g., blue, round
   - **n-ary** (relations defined for n terms)

     E.g., friends, siblings

3. Functions: Functions map their input object to the output object using their underlying relation.

Eg: father_of(), mother_of() etc.

First-Order Logic in Artificial intelligence

Parts of First Order Logic

First-order logic in Artificial Intelligence comprises two main components, which are as follows:

1. Syntax: Decides which collection of symbols is a logical expression. The basic syntactic elements of FOL are symbols. We use symbols to write statements in shorthand notation.

2. Semantics: Semantics refers to the techniques that we use to evaluate an expression of First Order Logic in AI. These techniques use various known relations and facts of the respective environment to deduce the boolean value of the given First Order Logic expression.

First-Order Logic in Artificial intelligence

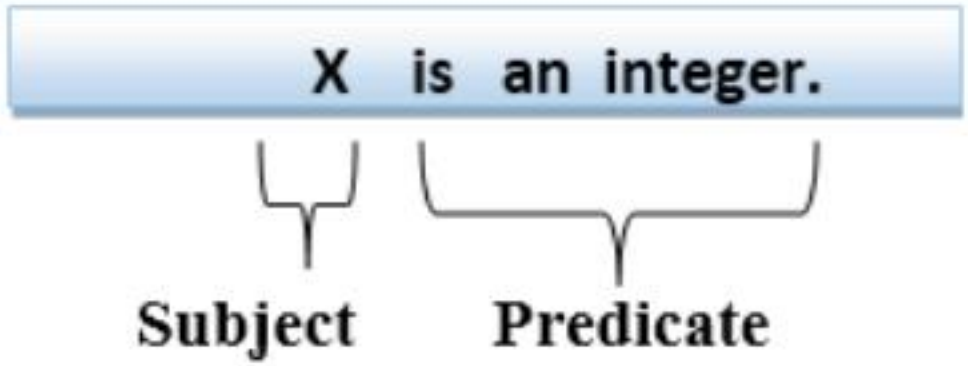
Basic elements of FOL

Name	Symbol
Constant	1, 6, A,W,New York, Elie, Dog...
Variables	a, b, c, x, y, z...
Predicates	<, >, brother, sister, father...
Equality	==
Function	Sqrt, LessThan, Sin(θ)...
Quantifier	∀, ∃
Connectives	∧, ∨, ¬, ⇒, ⇔

First-Order Logic in Artificial intelligence

FOL is further divided into two parts:

- **Subject**: the main part of the statement.
- **Predicate**: defined as a relation that binds two atoms together in a statement.
- **Consider the statement: "x is an integer."**, it consists of two parts, the first part x is the subject of the statement and second part "is an integer," is known as a p r e d i c a t e .



First-Order Logic in Artificial intelligence

Quantifiers in First-Order Logic

- **Quantifiers** in First Order Logic are used to quantify any entity in a given environment.
- Quantifiers allow us to determine or identify the range and scope of the variable in a logical expression.
- **Quantification** refers to the identification of the total number of an entity that is present in the environment and satisfies a given expression in First Order Logic in Artificial Intelligence.
- There are two types of quantifiers:
  - **Universal quantifier**: for all, everyone, everything.
  - **Existential quantifier**: for some, at least one.

First-Order Logic in Artificial intelligence

Universal Quantifier

- Universal quantifier is a symbol of logical representation, which specifies that the statement within its range is true for everything or every instance of a particular thing.
- The Universal quantifier is represented by a symbol ∀, which resembles an inverted A.
- If x is a variable, then ∀x can read as:
  - *For all x*
  - *For every x*
  - *For each x*

First-Order Logic in Artificial intelligence

Examples FOL using Universal quantifier :

1. All man drink coffee.

   $\forall x$ man(x) → drink (x, coffee).

2. Every Student Likes Educative.

   $\forall x$ student(x) → likes(x, Educative).

3. All cats like fish.

   $\forall x$ cat (x) ⟹ likes (x, fish).

4. All birds fly.

   $\forall x$ bird(x) → fly(x).

5. Every man respects his parents.

   $\forall x$ man(x) → respects (x, parent).

6. Not all students like both Mathematics and Science.

   $\neg \forall$ (x) [student(x) → like(x, Mathematics) $\wedge$ like(x, Science)].

First-Order Logic in Artificial intelligence

Existential Quantifier

- Existential quantifiers are the type of quantifiers, which express that the statement within its scope is true for at least one instance of something.
- It is denoted by the logical operator ∃, which resembles as inverted E. When it is used with a predicate variable then it is called as an existential quantifier.
- If x is a variable, the existential quantifier will be ∃x:
  - For some x
  - There exists an x
  - For at least one x

First-Order Logic in Artificial intelligence

Examples FOL using Existential quantifier :

1. Some boys are intelligent.

$\exists$ x: boys(x) $\wedge$ intelligent(x).

2. Some people like Football.

$\exists$ x: people(x) $\wedge$ likes Football(x)

3. Some boys play cricket.

$\exists$ x boys(x) $\rightarrow$ play(x, cricket).

4. Some students like ice cream

$\exists$ *x* student (*x*) $\wedge$ likes (*x*, ice-cream)

Continue…

Free and Bound Variables:

There are two types of variables in First-order logic which are given below:

- **Free Variable:** A variable is said to be a free variable in a formula if it occurs outside the scope of the quantifier.

  Example: ∀x ∃(y)[P (x, y, z)], where z is a free variable.

- **Bound Variable:** A variable is said to be a bound variable in a formula if it occurs within the scope of the quantifier.

  Example: ∀x ∃(y)[P (x, y, z)], where x and y are a bound variable.

Inference in First-Order Logic

- Inference in First-Order Logic is used to deduce new facts or sentences from existing sentences. Before understanding the FOL inference rule, let's understand some basic terminologies used in FOL.

Continue…

Substitution:

- Substitution is a fundamental operation performed on terms and formulas. It occurs in all inference systems in first-order logic. The substitution is complex in the presence of quantifiers in FOL. If we write **F[a/x]**, so it refers to substitute a constant "**a**" in place of variable "**x**".

- ***Note: First-order logic is capable of expressing facts about some or all objects in the universe.***

Continue…

Equality:

- First-Order logic does not only use predicate and terms for making atomic sentences but also uses another way, which is equality in FOL. For this, we can use **equality symbols** which specify that the two terms refer to the same object.

Example: Brother (John) = Smith.

- As in the above example, the object referred by the **Brother (John)** is similar to the object referred by **Smith**. The equality symbol can also be used with negation to represent that two terms are not the same objects.

Example: ¬(x=y) which is equivalent to x ≠y.

Continue…

FOL inference rules for quantifier:

As propositional logic we also have inference rules in first-order logic, so following are some basic inference rules in FOL:

- **Universal Generalization**
- **Universal Instantiation**
- **Existential Instantiation**
- **Existential introduction**

Continue…

1. Universal Generalization:

- Universal generalization is a valid inference rule which states that if premise P(c) is true for any arbitrary element c in the universe of discourse, then we can have a conclusion as ∀ x P(x).

- It can be represented as: $$\frac{P(c)}{\forall x\, P(x)}$$

- This rule can be used if we want to show that every element has a similar property.

- In this rule, x must not appear as a free variable.

- **Example:** Let's represent, P(c): "**A byte contains 8 bits**", so for **∀ x P(x)** "**All bytes contain 8 bits**.", it will also be true.

Continue…

2. Universal Instantiation:

- Universal instantiation is also called as universal elimination or UI is a valid inference rule. It can be applied multiple times to add new sentences.
- The new KB is logically equivalent to the previous KB.
- As per UI, **we can infer any sentence obtained by substituting a ground term for the variable**.
- The UI rule state that we can infer any sentence P(c) by substituting a ground term c (a constant within domain x) from **∀ x P(x) for any object in the universe of discourse**.

- It can be represented as:

$$\frac{\forall x\, P(x)}{P(c)}$$

Example:1.

- IF "Every person like ice-cream"=> ∀x P(x) so we can infer that "John likes ice-cream" => P(c)

Continue…

- **Example: 2.**

  Let's take a famous example,

  "All kings who are greedy are Evil." So let our knowledge base contains this detail as in the form of FOL:

  $\forall x$ king(x) $\wedge$ greedy (x) $\rightarrow$ Evil (x),

- So from this information, we can infer any of the following statements using Universal Instantiation:

  King(John) $\wedge$ Greedy (John) $\rightarrow$ Evil (John),

  King(Richard) $\wedge$ Greedy (Richard) $\rightarrow$ Evil (Richard),

  King(Father(John)) $\wedge$ Greedy (Father(John)) $\rightarrow$ Evil (Father(John)),

Continue…

3. Existential Instantiation:

- Existential instantiation is also called as Existential Elimination, which is a valid inference rule in first-order logic.
- It can be applied only once to replace the existential sentence.
- This rule states that one can infer P(c) from the formula given in the form of ∃x P(x) for a new constant symbol c.
- The restriction with this rule is that c used in the rule must be a new term for which P(c) is true.
- It can be represented as:

Example:

- From the given sentence: **∃x Crown(x) ∧ OnHead(x, John),**

Continue…

So we can infer: **Crown(K) ∧ OnHead(K, John),** as long as K does not appear in the knowledge base.

- The above used K is a constant symbol, which is called **Skolem constant**.
- The Existential instantiation is a special case of **Skolemization process**.

Continue…

4. Existential introduction

- An existential introduction is also known as an existential generalization, which is a valid inference rule in first-order logic.

- This rule states that if there is some element c in the universe of discourse which has a property P, then we can infer that there exists something in the universe which has the property P.

- It can be represented as: $$\frac{P(c)}{\exists x P(x)}$$

- **Example: Let's say that,**
  "Priyanka got good marks in English."
  "Therefore, someone got good marks in English."

Monkey and Banana Problem

Problem Statement

Suppose the problem is as given below –

- A hungry monkey is in a room, and he is near the door.
- The monkey is on the floor.
- Bananas have been hung from the center of the ceiling of the room.
- There is a block (or chair) present in the room near the window.
- The monkey wants the banana, but cannot reach it.

Monkey and Banana Problem

So if the monkey is clever enough, he can come to the block, drag the block to the center, climb on it, and get the banana. Below are few observations in this case –

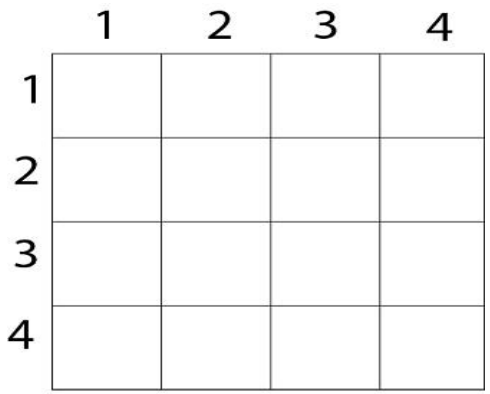
- Monkey can reach the block, if both of them are at the same level. From the above image, we can see that both the monkey and the block are on the floor.
- If the block position is not at the center, then monkey can drag it to the center.
- If monkey and the block both are on the floor, and block is at the center, then the monkey can climb up on the block. So, the vertical position of the monkey will be changed.
- When the monkey is on the block, and block is at the center, then the monkey can get the bananas.

Monkey and Banana Problem

- Now, let us see how we can solve this using Prolog. We will create some predicates as follows –
- We have some predicates that will move from one state to another state, by performing action.
- When the block is at the middle, and monkey is on top of the block, and monkey does not have the banana (i.e. ***has not*** state), then using the ***grasp*** action, it will change from ***has not*** state to ***has*** state.
- From the floor, it can move to the top of the block (i.e. ***on top*** state), by performing the action ***climb***.
- The **push** or **drag** operation moves the block from one place to another.
- Monkey can move from one place to another using **walk** or **move** clauses.
- Another predicate will be canget(). Here we pass a state, so this will perform move predicate from one state to another using different actions, then perform canget() on state2. When we have reached to the state '**has>**', this indicates '**has banana**'.
- We will stop the execution.

N-Queen Problem

- N - Queens problem is to place n - queens in such a manner on an n x n chessboard that no queens attack each other by being in the same row, column or diagonal.
- It can be seen that for n =1, the problem has a trivial solution, and no solution exists for n =2 and n =3. So first we will consider the 4 queens problem and then generate it to n - queens problem.



4x4 chessboard

- Since, we have to place 4 queens such as q_1 q_2 q_3 and q_4 on the chessboard, such that no two queens attack each other. In such a conditional each queen must be placed on a different row, i.e., we put queen "i" on row "i."
- Some possible solutions: 2, 4, 1, 3, and 3, 1, 4, 2

4-Queen Problem

- Possible solutions: 2, 4, 1, 3, and 3, 1, 4, 2

	1	2	3	4
1			q_1	
2	q_2			
3				q_3
4		q_4		

8-Queen Problem

- Possible solution: 4,6,8,2,7,1,3,5

	1	2	3	4	5	6	7	8
1				q_1				
2						q_2		
3								q_3
4		q_4						
5							q_5	
6	q_6							
7			q_7					
8					q_8			

N-Queen Problem

- While there are algorithms like Backtracking to solve N Queen problem, let's take an AI approach in solving the problem.

- It's obvious that AI does not guarantee a globally correct solution all the time but it has quite a good success rate of about 97% which is not bad.
-
- A description of the notions of all terminologies used in the problem will be given and are as follows:
- **Notion of a State** – A state here in this context is any configuration of the N queens on the N X N board. Also, in order to reduce the search space let's add an additional constraint that there can only be a single queen in a particular column. A state in the program is implemented using an array of length N, such that if **state[i]=j** then there is a queen at column index **i** and row index **j**.

N-Queen Problem

- **Notion of Neighbors** – Neighbors of a state are other states with board configuration that differ from the current state's board configuration with respect to the position of only a single queen. This queen that differs a state from its neighbor may be displaced anywhere in the same column.

- **Optimization function or Objective function** – We know that local search is an optimization algorithm that searches the local space to optimize a function that takes the state as input and gives some value as an output. The value of the objective function of a state here in this context is the number of pairs of queens attacking each other. Our goal here is to find a state with the minimum objective value. This function has a maximum value of NC2 and a minimum value of 0.

N-Queen Problem

Algorithm:

- Start with a random state(i.e., a random configuration of the board).
- Scan through all possible neighbors of the current state and jump to the neighbor with the highest objective value, if found any. If there does not exist, a neighbor, with objective strictly higher than the current state but there exists one with equal then jump to any random neighbor(escaping shoulder and/or local optimum).
- Repeat step 2, until a state whose objective is strictly higher than all it’s neighbor's objectives, is found and then go to step 4.
- The state thus found after the local search is either the local optimum or the global optimum. There is no way of escaping local optima but adding a random neighbor or a random restart each time a local optimum is encountered increases the chances of achieving global optimum(the solution to our problem).
- Output the state and return.

N-Queen Problem

- It is easily visible that the global optimum in our case is 0 since it is the minimum number of pairs of queens that can attack each other. Also, the random restart has a higher chance of achieving global optimum, but we still use random neighbor because our problem of N queens does not have a high number of local optima and random neighbor is faster than random restart.
-
- **Conclusion:**
  - **Random Neighbor** escapes shoulders but only has a little chance of escaping local optima.
  - **Random Restart** both escapes shoulders and has a high chance of escaping local optima.

Traveling Salesman Problem (TSP)

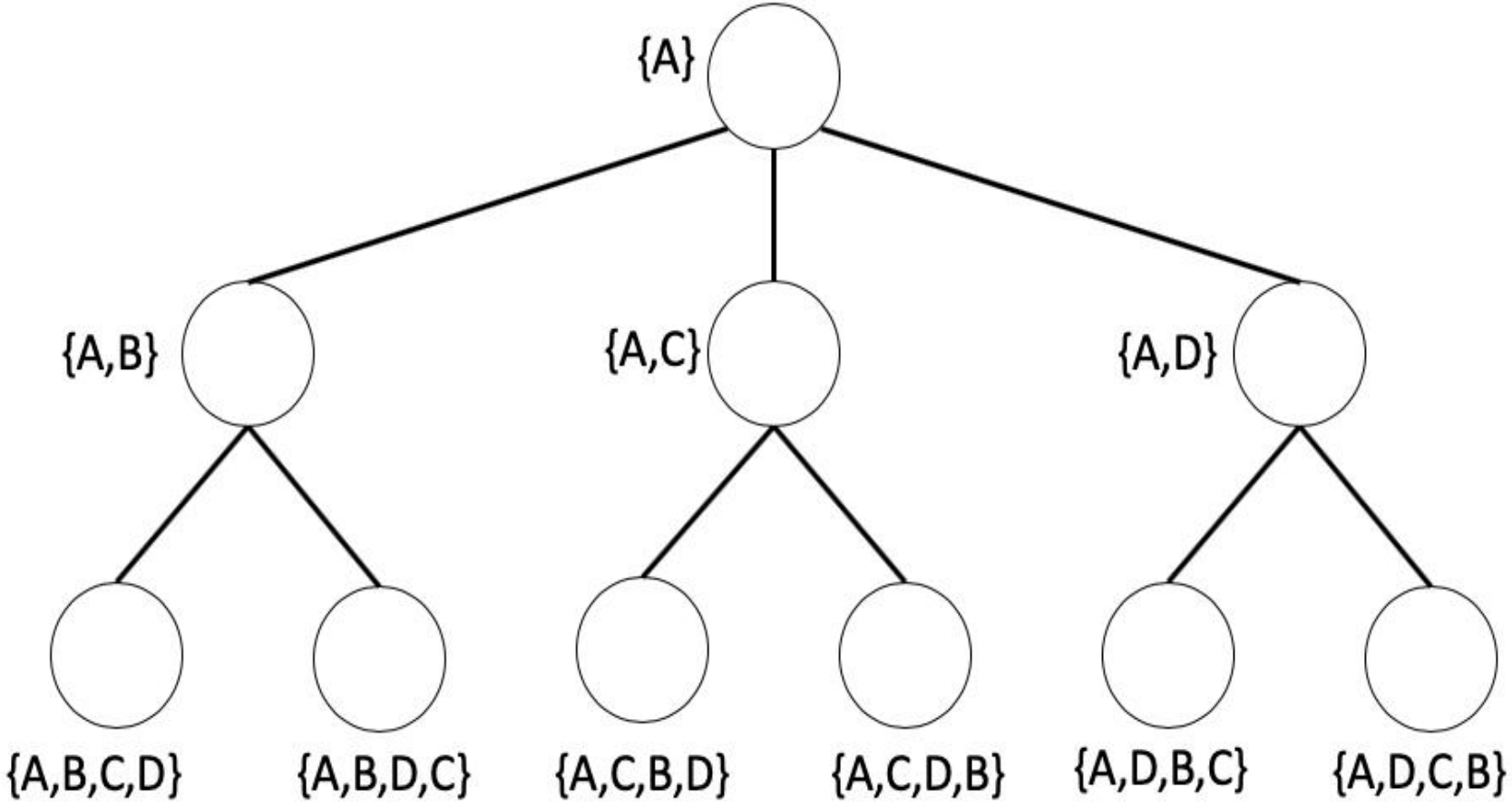
- Consider the following situation. You are given a list of n cities with the distance between any two cities. Now, you have to start with your office and to visit all the cities only once each and return to your office. What is the shortest path can you take? This problem is called the Traveling Salesman Problem (TSP).

Problem Formulation of TSP

- To make the problem simple, we consider 3-city-problem. Let's call the office (A) and the 3 cities (B) (C) (D) respectively. We initialize the problem state by {A} means the salesman departed from his office. As an operator, when he visited city-B, the problem state is updated to {A, B}, where the order of elements in { } is considered. When the salesman visited all the cities, {A, B, C, D} in this case, the departed point A is automatically added to the state which means {A, B, C, D, A}. Therefore, the initial state of this TSP is {A} and the final state(goal) is {A, X1, X2, X3, A} where traveled distance is minimized.

- Taking each state as a node of a tree structure, we can represent this TSP as the following tree search problem.

Example



Predicate Logic in AI (Artificial Intelligence)

Predicate logic, also known as **first-order logic (FOL)**, extends propositional logic by allowing AI systems to **represent relationships between objects** and their **properties**. This makes predicate logic a powerful tool for **knowledge representation and reasoning**. It enables AI systems to understand relationships like “John is the father of Mary” or “All humans are mortal.”

Predicate Logic in AI (Artificial Intelligence)

Predicate logic, also known as **first-order logic (FOL)**, is an extension of propositional logic that allows us to express **relationships between objects** and their **properties**. In AI, predicate logic is widely used to represent **knowledge** and perform **reasoning** in more complex scenarios where relationships matter.

Unlike propositional logic, which deals with simple true/false statements, predicate logic introduces **predicates, variables, constants, and quantifiers**. These elements help in modeling real-world problems that involve **multiple objects** and their **interactions**.

Role of Predicate Logic in AI

- **Knowledge Representation:** It provides a structure for representing complex facts about objects and their relationships in a system.
- **Reasoning:** AI systems use predicate logic to **infer new information** from existing facts, making it suitable for **decision-making tasks**.

Example:

- “John is the father of Mary” can be represented as: Father(John,Mary)Father(John, Mary)Father(John,Mary)

This expression tells us that there is a **relationship** (Father) between two objects (John and Mary). Predicate logic enables us to model and reason about such relationships effectively.

Components of Predicate Logic:

Predicate logic involves several key components that allow it to **represent relationships** and **properties** of objects in a structured way.

1. Predicates:

- A **predicate** is a function that returns either **true** or **false** based on the relationship between its arguments.
- Example: IsHungry(John)IsHungry(John)IsHungry(John) This predicate represents whether John is hungry, returning true if he is and false if not.

2. Variables:

- **Variables** are placeholders for objects within a domain. They allow us to represent general statements that apply to multiple objects.
- Example: In **IsHungry(x)**, the variable **x** can represent any person.

3. Constants:

- **Constants** represent specific objects or entities in the domain.
- Example: **John** is a constant in the predicate **IsHungry(John)**.

Structure of Predicates

Predicate consists of two key elements: the **predicate symbol** and **arguments**. Predicates are enhanced with **quantifiers** to specify the **scope** of variables involved.

1. Predicate Symbol

- The **predicate symbol** defines the **property** or **relationship** being described.
- Example:
  - **IsHungry(x)** represents whether a person (x) is hungry.
  - **Married(x, y)** denotes that person **x** is married to person **y**.
- Predicates are named based on the relationship or property they represent. The symbol is followed by **arguments** enclosed in parentheses.

2. Arguments and Arity

- **Arguments** refer to the specific **objects** that the predicate is applied to.
- The **arity** of a predicate refers to the **number of arguments** it takes.

.

Examples:

- **IsHungry(x)**: A predicate with **1 argument** (arity = 1).
- **Married(x, y)**: A predicate with **2 arguments** (arity = 2).

X(a, b, c): A predicate with **3 arguments** (arity = 3), representing something like "a + b + c = 0."

3. Quantifiers in Predicate Logic

Quantifiers allow us to **specify the scope** of variables. There are two main types:

1. **Existential Quantifier (∃)**
   - **Meaning:** There exists at least **one object** that satisfies the given condition.
   - **Example:** ∃x IsHungry(x)
     This statement means that **at least one person** is hungry.
   - **Negation:** The negation of the existential quantifier means that no such object exists.
     ¬∃x IsHungry(x)
   - This means that **no one is hungry**.

2. Universal Quantifier ($\forall$)

- **Meaning:** The given condition holds **for all objects** in the domain.
- **Example:** $\forall x\ (IsHuman(x) \rightarrow IsMortal(x))$
  This means that **all humans are mortal**.
- **Negation:** The negation of the universal quantifier means there is **at least one exception**.
  $\neg \forall x\ IsHuman(x) \rightarrow IsMortal(x)$
  This implies that **at least one human is not mortal**.

Knowledge Representation using Predicate Logic in AI

Knowledge representation is a critical part of AI, enabling systems to **store, reason, and derive new facts** from existing information. **Predicate logic** offers a structured way to represent **complex relationships, facts, and rules** in a logical and understandable format.

How Predicate Logic Helps in Knowledge Representation

1. **Representing Relationships Between Objects:**

   - Example: Married(John,Mary)
   - This expression indicates that John is married to Mary. Predicate logic captures such **binary relationships** between objects, helping AI systems understand connections.

2. **Storing Rules in Knowledge Bases:**

   - Rules can be defined using predicates and quantifiers.
   - **Example: $\forall x$ (IsHuman(x)→IsMortal(x))** This rule states that all humans are mortal. AI systems can **store** these rules and apply them during reasoning.

3. Handling Multiple Entities and Properties:

- AI systems can store **complex information** by using predicates with multiple arguments.
- **Example: Parent(John,Mary)∧Sibling(Mary,Kate)** This expresses that John is Mary's parent and Mary is Kate's sibling.

4.Reasoning and Inference:

- AI systems can use **inference rules** to derive new facts from stored knowledge.
- Example: If the system knows that: **∀x (Parent(John,x) →Loves(John,x))** It can infer that John loves all his children.

Difference Between Predicate Logic and Propositional Logic

Aspect	Predicate Logic	Propositional Logic
Definition	Extends propositional logic by allowing variables and relationships between objects.	Deals with simple statements that are either true or false.
Elements	Uses predicates, variables, constants, and quantifiers.	Uses atomic propositions (simple statements).
Expressiveness	More expressive; can represent relationships and properties.	Less expressive; limited to simple true/false statements.
Example	$\forall x$ (IsHuman(x) $\rightarrow$ IsMortal(x))	It is raining (P).
Quantifiers	Supports existential ($\exists$) and universal ($\forall$) quantifiers.	No quantifiers are used.
Handling of Relationships	Can represent complex relationships, like Parent(John, Mary).	Cannot represent relationships between objects.
Applications	Used in NLP, expert systems, and robot planning.	Used in decision-making systems with simpler conditions.
Complexity	More complex due to the use of variables and quantifiers.	Simpler with fewer components.
Reasoning Capability	Enables inference through rules and relationships.	Limited to evaluating truth values of propositions.